Efficient LLM Serving on a Single GPU

Project Report · CS5903 Distributed AI Training, Fall 2026

Group 4, Project 24

Vishank Singh AI23BTECH11032
Anup Chavan CS23BTECH11005

Code and data: https://github.com/anupchavan/efficient-llm-serving

1 Problem

In default vLLM, every decode pass reads the full 16-bit weights to produce one token per request, and most of the GPU's compute sits idle during decode. Known fixes are smaller weights, a smaller KV cache, reuse of prompt prefixes that were already processed, and generating more than one token per model pass.

These fixes interact. Quantisation frees memory, while the draft model used for speculative decoding takes memory and GPU time. Whether a fix helps also depends on the load on the server, the prompt length and how much prompts repeat. Hence speculative decoding, for example, can help at light load and hurt at high load.

The project goal is to improve the latency and throughput of a served open-weight model with these techniques, and to find which of them give the gain and under which conditions. The milestones are:

Milestone	Scope	Expected outcome
M1	Quantised weights, FP8 KV cache, batching on one GPU	Measured gain over default vLLM at a latency target, with output quality kept
M2	Prefix caching, speculative decoding	Conditions where each helps; the load where speculative decoding stops paying
M3	All techniques combined	Contribution of each technique; the limiting resource; regimes where the chosen setup is wrong

2 How a request is served

A request first waits in a queue. When it gets a slot, the model runs **prefill**: it processes the whole prompt in one pass. Then it runs **decode**: one new token per pass, until the answer is complete.

During both phases the model stores the attention keys and values of every token in GPU memory (the **KV cache**), so that each new token does not have to recompute the whole sequence. The KV cache grows with every token, so its size limits how many requests can run at the same time.

The two phases are limited by different resources. Every decode pass reads all the model weights from GPU memory to produce a single token per request, so decode is limited by memory bandwidth. Prefill processes many tokens per pass, so it is limited by compute.

We measure:

- **TTFT** (time to first token): queue time plus prefill.
- **TPOT** (time per output token): average time for each token after the first.
- **Throughput**: requests finished per second.

3 Techniques

Technique	Idea	Bottleneck	vLLM flag
AWQ [5]	Weights stored in 4 bits instead of 16	Memory bandwidth in decode	`-quantization awq`
FP8 KV cache	KV cache stored in 8 bits; about twice as many tokens fit	KV-cache capacity	`-kv-cache-dtype fp8`
Prefix caching [1, 3]	KV cache of a prompt prefix seen before is reused	Prefill compute	`-enable-prefix-caching`
Speculative decoding [4]	A 0.5B draft model guesses 5 tokens; the 3B model verifies them in one pass	Idle compute in decode	`-speculative-config`

Speculative decoding does not change the output, because the 3B model verifies every token. AWQ changes the weights, so output quality needs a separate check.

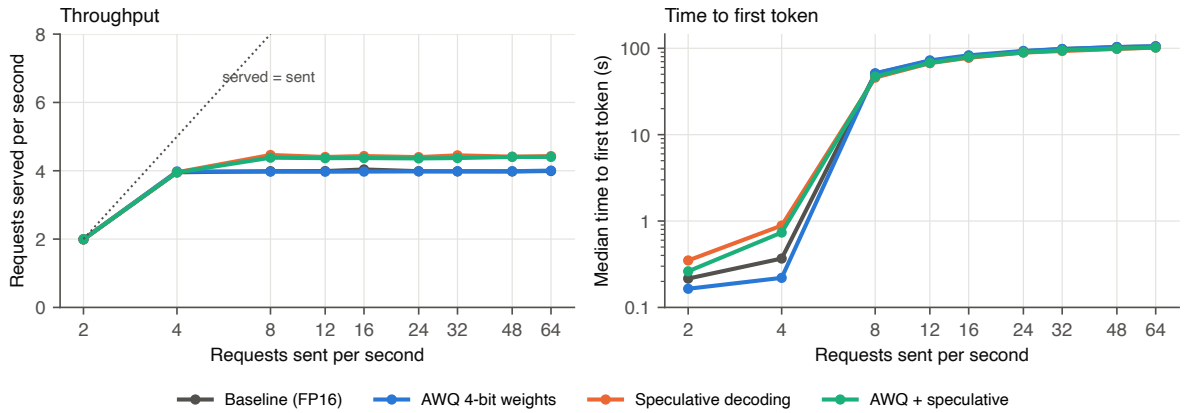
4 Approach

We switch each technique on alone and then in combination, so that the effect of each technique can be separated from the interactions. Each setting is run under a range of loads, from light to far past what the GPU can handle.

Item	Setting
GPU	NVIDIA RTX PRO 4000 Blackwell, 24 GB
Server	vLLM 0.30 [1] with PagedAttention and chunked prefill [2] (both on by default), FlashAttention backend, `-gpu-memory-utilization 0.95`, `-max-model-len 8192`
Models	Qwen2.5-3B-Instruct (FP16) or Qwen2.5-3B-Instruct-AWQ; Qwen2.5-0.5B as draft model
Configurations	Baseline (vLLM defaults), AWQ, speculative decoding, AWQ + speculative decoding
Load generator	`vllm bench serve`, 1,000 requests per run, Poisson arrivals
Workload	1,024 prompt tokens and 128 output tokens per request
Request rates	2, 4, 8, 12, 16, 24, 32, 48, 64 requests/s
Repeats	Up to 6 per setting; 201 completed runs in total

5 Results

5.1 Throughput limit

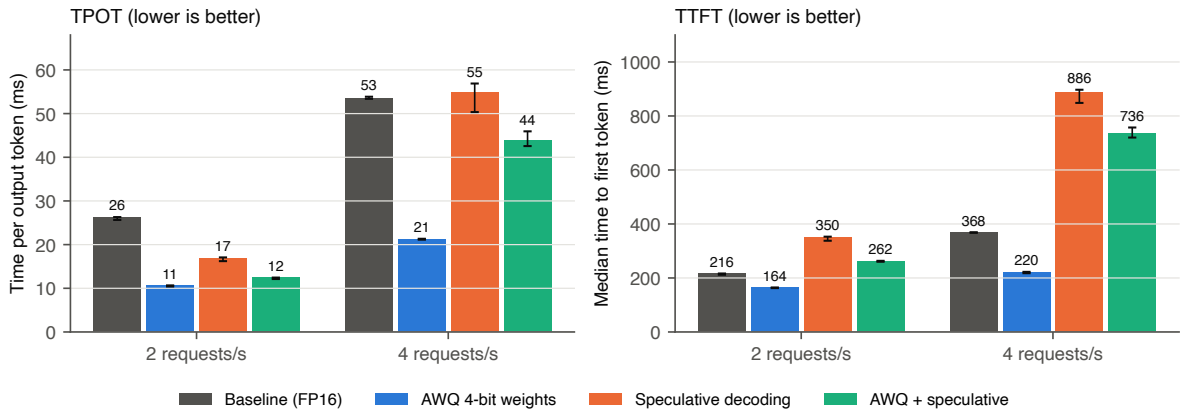


Up to 4 requests/s, every request is served and the median TTFT stays between 0.16 and 0.9

s. Above that, the server finishes 4.0–4.4 requests/s regardless of how many are sent. The rest wait in the queue, and the median TTFT reaches 50–106 s.

None of the techniques moves this limit much. At 4 requests/s the GPU processes about 4,000 prompt tokens/s, so we think the limit is prefill compute. This still has to be confirmed by profiling. Everything above 4 requests/s is an overloaded server, and the latency numbers there are mostly queueing time.

5.2 Below the limit



Bars show the median of 4–6 runs, and the error bars show the range.

AWQ lowers TPOT about 2.5 times, from 26 to 11 ms at 2 requests/s and from 53 to 21 ms at 4 requests/s. TTFT is also lower at both loads. This matches decode being limited by memory bandwidth, since the AWQ model reads about a third as much weight data per token.

Speculative decoding lowers TPOT 1.6 times at 2 requests/s, but gives no gain at 4 requests/s. TTFT is higher at both loads, because the draft model also has to run prefill on the prompt.

5.3 Contribution of each technique

The table shows TPOT speed-up over the baseline, with the change in TTFT in brackets. The last column shows requests served per second when the server is overloaded.

Configuration	2 requests/s	4 requests/s	Overloaded, 16–64/s
Baseline (FP16, vLLM defaults)	1.00×	1.00×	4.0
AWQ	**2.47**× (−24%)	**2.52**× (−40%)	4.0
Speculative decoding	1.56× (+62%)	0.98× (+141%)	**4.4**
AWQ + speculative decoding	2.11× (+21%)	1.22× (+100%)	4.4

Combining the two does not add the gains: AWQ alone (2.5×) is faster than AWQ with speculative decoding (2.1× and 1.2×). With AWQ, each pass of the 3B model is already cheap, while the draft model is not quantised and still runs 5 steps per round. About 55% of the drafted tokens are accepted, i.e. around 3.7 tokens per verification pass, and this is no longer enough to pay for the draft steps.

Speculative decoding stops paying between 2 and 4 requests/s. At light load the GPU has idle compute during decode, which the draft model uses. At higher load the other requests in the batch already keep the GPU busy, so drafting and verification compete with them.

When overloaded, speculative decoding serves about 11% more requests per second (4.4 vs. 4.0). However, TTFT is over a minute in every configuration at that point, so this gain is of little use in practice. AWQ gives no throughput gain when overloaded. This fits a prefill limit, because AWQ reduces the weight data read per pass but not the compute.

6 Status

Item	Status	Note
M1: AWQ	Done	2.5× lower TPOT below 4 requests/s; output quality not yet measured
M1: FP8 KV cache	Blocked	Needs the CUDA toolkit on the server
M1: batching	Not started	Single GPU, so the choice is batch size and scheduling, not sharding
M2: speculative decoding	Done	Break-even between 2 and 4 requests/s
M2: prefix caching	Next	Needs prompts with shared prefixes
M3: contribution of each technique	First version	One prompt length only

7 Limitations and next steps

- All runs use one prompt length (1,024 tokens) and prompts with no shared prefix.
- The load generator sends requests at a fixed rate, so loads above the limit only measure queueing.
- AWQ output quality is not measured yet.
- One GPU and one model size.

Next two weeks	Then	Before the final review
Prompt lengths 512–6,144	Fixed-concurrency sweep	Contribution of each technique across regimes
Prefix caching off vs. on, with shared prefixes	2–3 draft tokens; other draft methods	Best setup per regime
CUDA toolkit; FP8 KV cache	Goodput at the latency target	Profiling of the limit
AWQ quality on a small benchmark	Batch-size and scheduling tuning	Final plots and scripts

For M1 we use TTFT under 1 s and TPOT under 50 ms as the latency target, and will report goodput: the requests per second that meet both.

8 Repository

`https://github.com/anupchavan/efficient-llm-serving` contains the benchmark script (`scripts/`), the 201 raw result files (`data/raw/`), a per-run summary table (`data/summary.csv`), the analysis and plotting scripts (`analysis/`), the figures, this report and the slides.

References

[1] Efficient Memory Management for Large Language Model Serving with PagedAttention — Kwon et al., SOSP 2023. https://arxiv.org/abs/2309.06180

[2] Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve — Agrawal et al., OSDI 2024. https://arxiv.org/abs/2403.02310

[3] SGLang: Efficient Execution of Structured Language Model Programs — Zheng et al., NeurIPS 2024. https://arxiv.org/abs/2312.07104

[4] Fast Inference from Transformers via Speculative Decoding — Leviathan et al., ICML 2023. https://arxiv.org/abs/2211.17192

[5] AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration — Lin et al., MLSys 2024. https://arxiv.org/abs/2306.00978